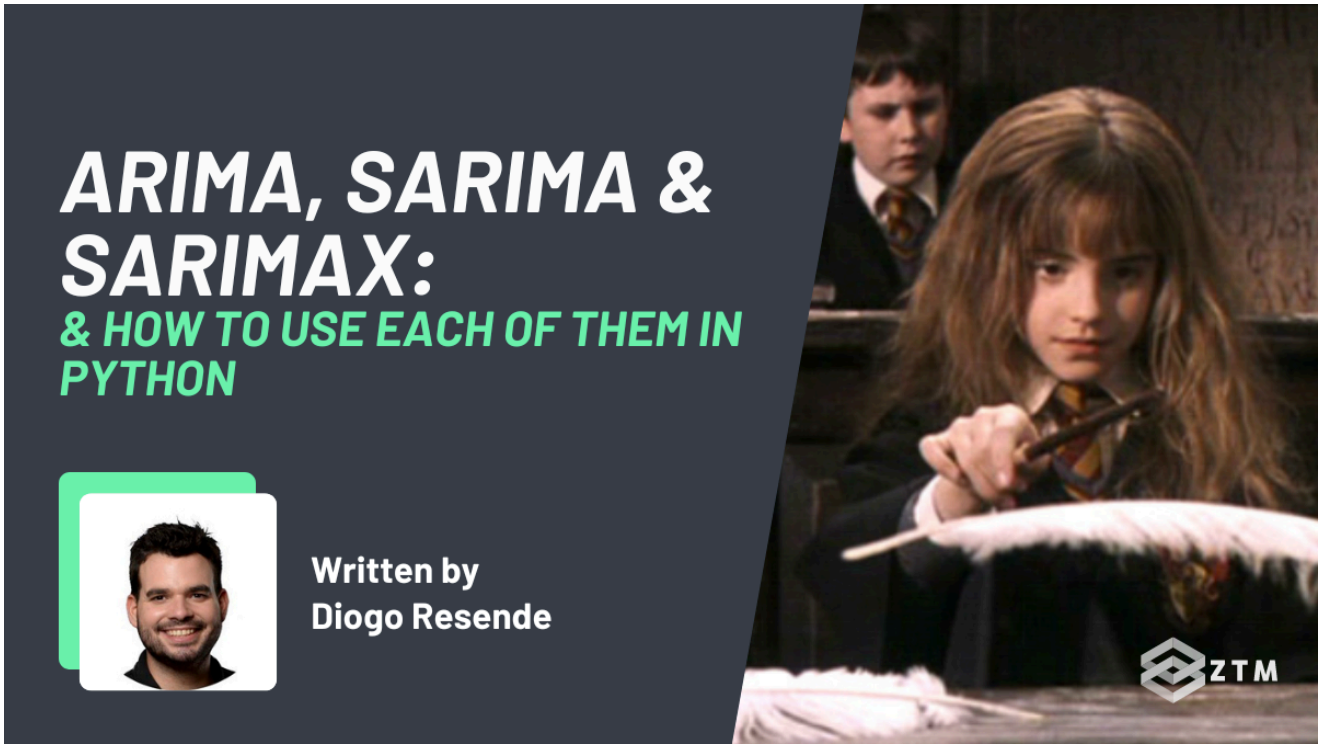
ARIMA, SARIMA, and SARIMAX Explained

zerotomastery.io/blog/arima-sarima-sarimax-explained/



Are you looking for a time forecasting tool that's as reliable as Hermione Granger's foresight? Then look no further than SARIMAX!

Just like Hermione, SARIMAX has a knack for seeing things before they happen. It can identify patterns in data and use them to predict what's coming next.

But unlike Hermione, SARIMAX doesn't require hours of studying in the Hogwarts library or a Time-Turner to get the job done. It's a powerful yet accessible forecasting model that anyone can use to gain insight into the future.

However, if you've just started looking into SARIMAX as a tool, you've probably been confused by the multiple similarly named models, such as ARIMA and SARIMA.

This leads to common questions such as:

- What's the difference between ARIMA, SARIMA, and SARIMAX?
- Are they all the same thing?
- Are they updated versions?
- Is it just a typo?
- Is one model better than the other for certain tasks?
- Also, how do they work?

Don’t worry! By the end of this guide, I’ll walk you through the answers to each of these questions, as well as show you how to use each model in Python, with code examples.

So let’s dive in!

The History of ARIMA

ARIMA was first introduced by Box and Jenkins in the 1970s and has since become one of the most widely used models for time series forecasting.

At the time, most models were limited in their ability to account for the complexity and variability of time series data, so the model quickly gained popularity and became a standard tool for time series forecasting in various industries, including finance, economics, and marketing.

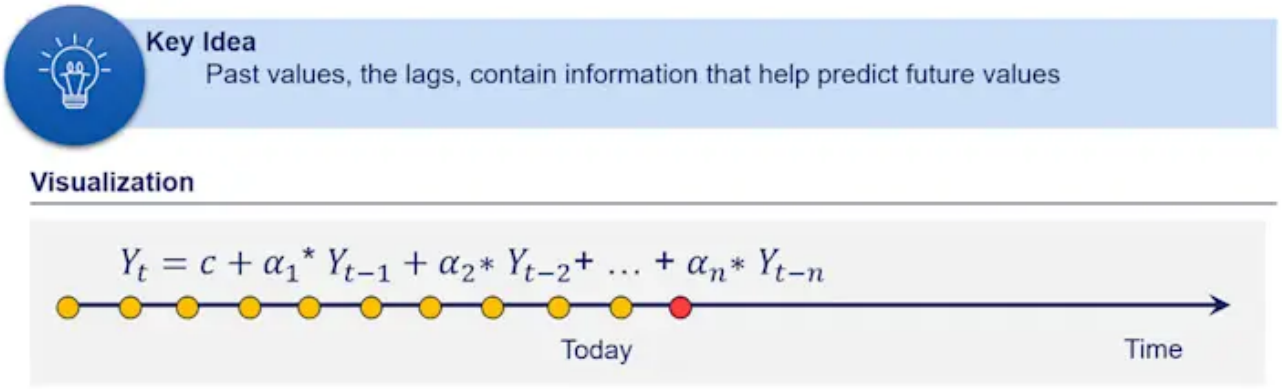
The 3 core components of ARIMA

ARIMA has three components, which I'll briefly introduce here:

1. Autoregression

The first is ‘autoregression’, which refers to the model's regression on its own lagged values.

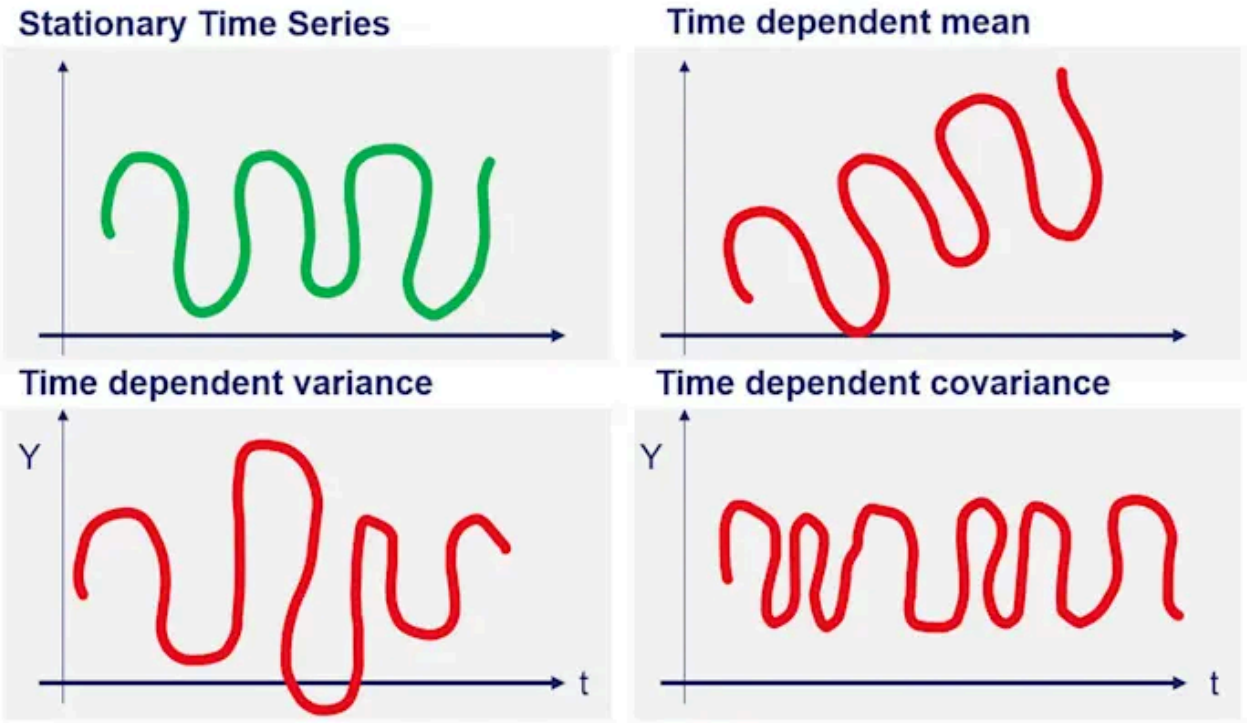
In simple terms, this means that **we use past data to predict future outcomes**.



2. Integrated

The second component is ‘integrated’, which deals with stationary time.

A stationary time series has a constant mean, variance, and covariance over time, which makes it easier to predict patterns.

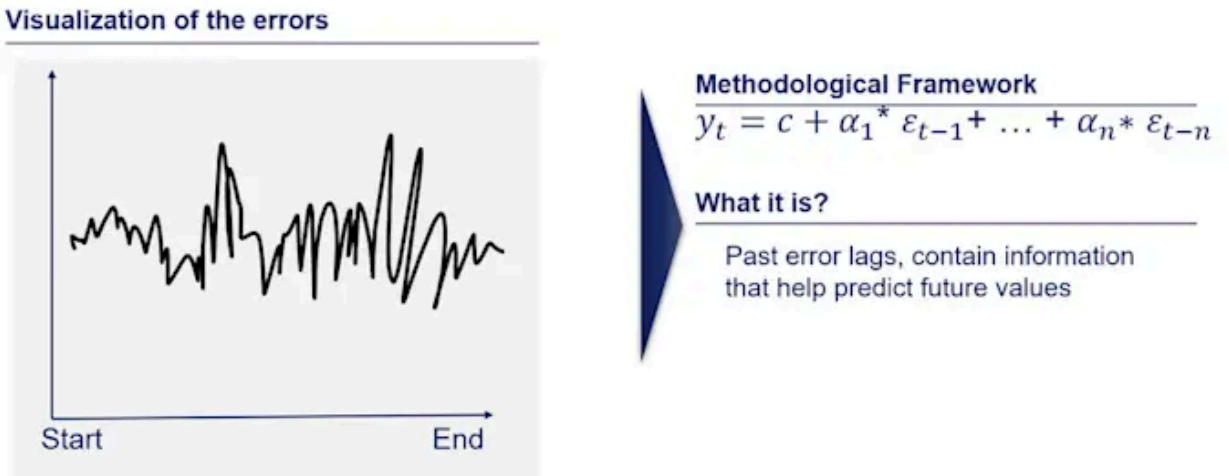


3. Average

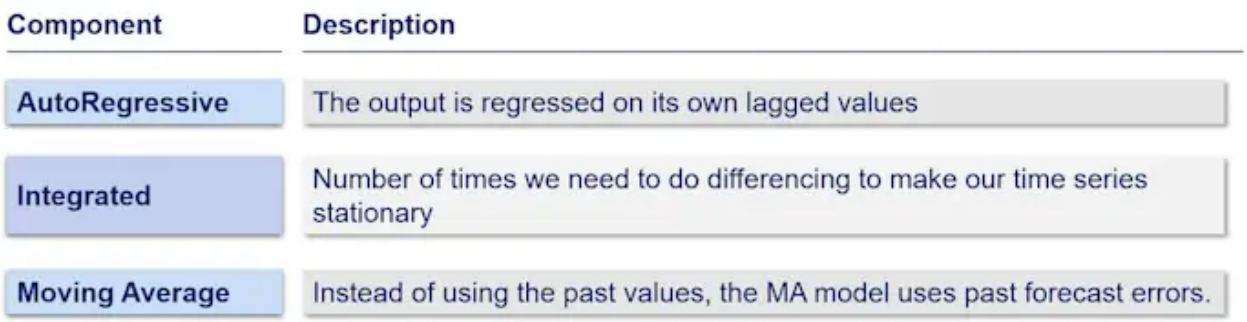
Finally, the third component is the moving ‘average’, which also uses past information but in a different way.

How is this different from autoregression?

Well, while autoregression uses past values of the time series, moving average uses the model's errors as information.



In summary, let’s recap with the following visualization:



Component	Description
AutoRegressive	The output is regressed on its own lagged values
Integrated	Number of times we need to do differencing to make our time series stationary
Moving Average	Instead of using the past values, the MA model uses past forecast errors.

Model selection in ARIMA

The ARIMA model also has three components: p, d, and q, which stand for "autoregressive", "differencing", and "moving average", respectively.

- The p-component (AR) measures the correlation between the current value of a time series and the values that came before it
- The d-component (I) represents the number of times the series needs to be differenced to make it stationary
- The q-component (MA) measures the correlation between the current value of the series and the errors from past predictions

To select the optimal values for these components, you need to use cross-validation and parameter tuning, which we will go deeper into, later in this post.

However, for a quick and easy solution, you can also use the `auto_arima` function from the `pdmarima` library in Python.

How to use ARIMA in Python

Now that you know the ARIMA fundamentals, let me show you how to apply ARIMA in Python, using the `pmdarima` library.

Let's import it, alongside the pandas and numpy libraries.

```
import numpy as np
import pandas as pd
from pmdarima import auto_arima
```

The next step is to get some data to work with.

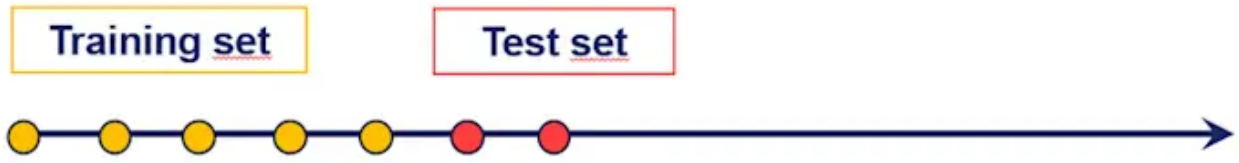
For this example, we'll generate some dummy data using `import random`.

```
import random

num_months = 60

dates = pd.date_range(start='2023-02-01', periods=num_months, freq='M')

values = [random.randint(1000, 10000) for i in range(num_months)]

df = pd.DataFrame({'Date': dates, 'Value': values})

df = df.set_index('Date')

print(df.head())
```

One very important practice when building a time series forecasting model is splitting the data into a training set, and a test set.

You see, unlike normal machine learning models, Time Series data has a context.

This means that **the values of today are influenced by the values of yesterday** which will in turn **influence tomorrow's data**.



Therefore, the most common practice is to take a few time periods at the end to test your model.

How many periods should it be?

It depends on the problem you're working with.

If your model will be used to forecast the next 3 months, then the test data should also be 3 months. If it will be used to predict the next 6 months, then the test data should be 6 months.

For this tutorial, I am going to set the test data to 6 months:

```
train = data[:-6]
test = data[-6:]
```

Now, we are finally ready to create the first ARIMA model.

(I'll usually suppress any warnings and set the error messages to be ignored).

```
arima_model = auto_arima(train, seasonal=False)
```

Next, you need to test your model for its accuracy.

There are 2 simple steps:

1. Predict the same data length as the test data
2. Compare the predictions with the test data

The KPI that I will use to measure the accuracy is the mean squared error, which is great for penalizing errors from outliers.

```
from sklearn.metrics import mean_squared_error


predictions = arima_model.predict(n_periods=len(test))


mse = mean_squared_error(test, predictions)


print("Mean Squared Error:", mse)
```

And there you have it folks!

Building an ARIMA model is as easy as pie - well, maybe a double-layered chocolate and salted caramel pie with a crispy toffee crumble on top. But still, a pie.

ARIMA vs SARIMA

So what's the difference between ARIMA and SARIMA and why would we use it?

Well, as you might already know, seasonality is an important factor in forecasting. As the seasons change, they have a huge impact on your data.

For example

Christmas sales in a store vs January.

Unfortunately, ARIMA doesn't have a seasonal component, which is why SARIMA was developed.

It’s not the only difference though.

Remember the p,d, and q components from the ARIMA model?

To build a SARIMA model, you need to add both seasonal components and an extra differencing component.

- The seasonal components are the seasonal autoregressive (SAR) and seasonal moving average (SMA) components
- While the extra differencing component is for the seasonal component and is denoted by D

Now, to find the optimal component values, you will also need to use the Cross-Validation and Parameter Tuning. (Granted, a shortcut is the `auto_arima` function, but more on that in a second).

tl;dr

With the SARIMA model, you can capture both the non-seasonal and seasonal patterns in your data and build a forecasting model.

It’s like playing detective with your data, hunting for the optimal values of p, d, q, P, D, and Q to solve the mystery of forecasting your time series.

Let’s take a look at how…

How to use SARIMA in Python

To apply a SARIMA model, you can use the `auto_arima` function from `pmdarima` to automatically select the optimal SARIMA model for your data.

You can then use the trained data that we created above:

```
sarima_model = auto_arima(train, seasonal=True, m=12)
```

In this example, we're setting the seasonal parameter to True since we're building a SARIMA model, and setting the seasonal period to 12 (since we're using monthly data).

Then, to make the predictions and assess the accuracy, you run the following, which is the same as for the ARIMA model.

```
predictions = sarima_model.predict(n_periods=len(test))


mse = mean_squared_error(test, predictions)


print("Mean Squared Error:", mse)
```

See how easy it is to use?

SARIMAX vs SARIMA

SARIMA was a great step forward in time series forecasting but it still had some limitations.

One of these limitations was its inability to incorporate exogenous variables that could have an impact on the time series data.

To address this limitation, the SARIMAX (Seasonal ARIMA with eXogenous regressors) model was introduced.

This means that you can incorporate additional information, such as economic indicators, into your forecasting model to improve its accuracy, as well as seasonality.

SARIMAX has become a popular model for time series forecasting in various industries and has been widely adopted due to its ability to incorporate important variables that are not part of the time series data.

How to use SARIMAX in Python

SARIMAX is where things get really interesting, so let's see how it works.

First, let's get some new data.

We are going to modify the `train_data` and `test_data` variables to reflect the new split.

We'll also keep the same logic so that the last 6 observations are the test data, and the rest are the training data.

```
np.random.seed(1)
dates = pd.date_range('20220101', periods=50)
data = pd.Series(np.random.normal(size=50), index=dates)
exog_data = pd.DataFrame(np.random.normal(size=(50,2)), index=dates, columns=
['Exog_Var1', 'Exog_Var2'])


train_data = data[:-6]
test_data = data[-6:]
```

We then use the `auto_arima()` function from the `pmdarima` library to automatically select the optimal values for the SARIMAX model components, including the exogenous regressors.

We then set:

- The `exogenous` argument to the exogenous regressors data for the training set
- The `seasonal` argument to `True` to account for the seasonal component, and
- The `m` argument to `12` to specify the seasonal period

```
sarimax_model = auto_arima(train_data, exogenous=exog_data[:len(train_data)],
seasonal=True, m=12)
forecast = sarimax_model.predict(n_periods=len(test_data),
exogenous=exog_data[len(train_data):])
```

Done! Now we just need to assess the model.

However, the difference with the SARIMA model is that **you must include the regressors from the test data and use them to predict**.

Like so:

```
predictions= model.predict(n_periods=len(test_data), exogenous =
exog_data[len(train_data):])


mse = mean_squared_error(test, predictions)


print("Mean Squared Error:", mse)
```

And there you have it, a SARIMAX model trained and tested.

Cross-validation - What is it? Why should you do it? What types exist?

Cross-validation is a fundamental concept for forecasting because it provides credibility to our model.

The key idea is to repeat the experiment or the testing in different situations to make sure the model works and gives the known results.

However, if you remember from earlier, I said that one crucial aspect of time series is that it is data with context. This means that the data of yesterday influences today's and tomorrow’s values.

Therefore, **when you build a model, it is important that you try it through all seasonality cycles**.

What we will do is have numerous training and test sets. For instance, we will add the test set to the training and do it several times.

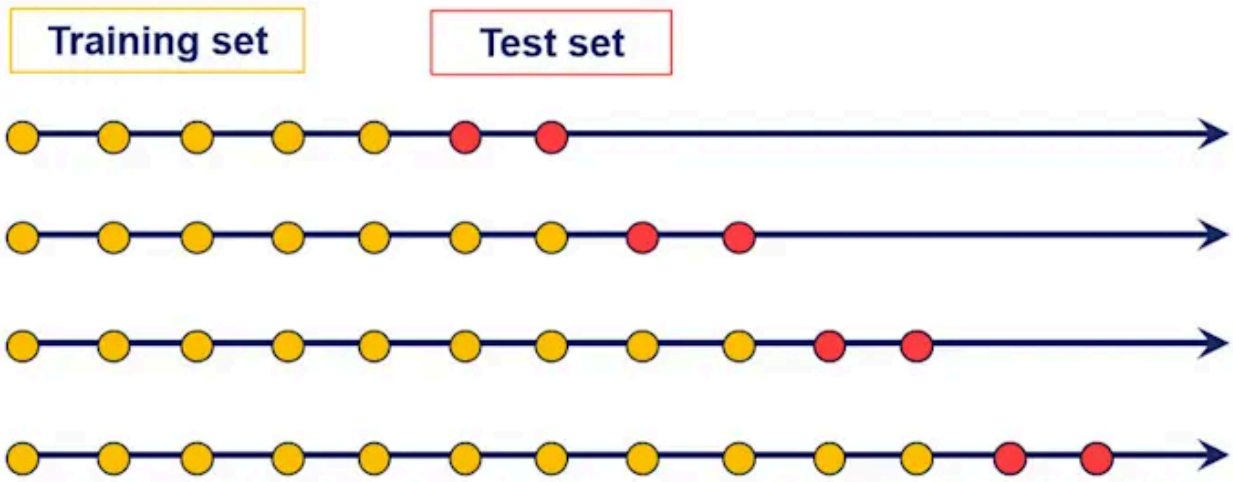
There are two types of Cross-Validation:

- Rolling forecasts, and
- Sliding forecasts

Rolling forecasts

In the following image, you can see that each time we add the test data to the training data, as we continue to validate the model.

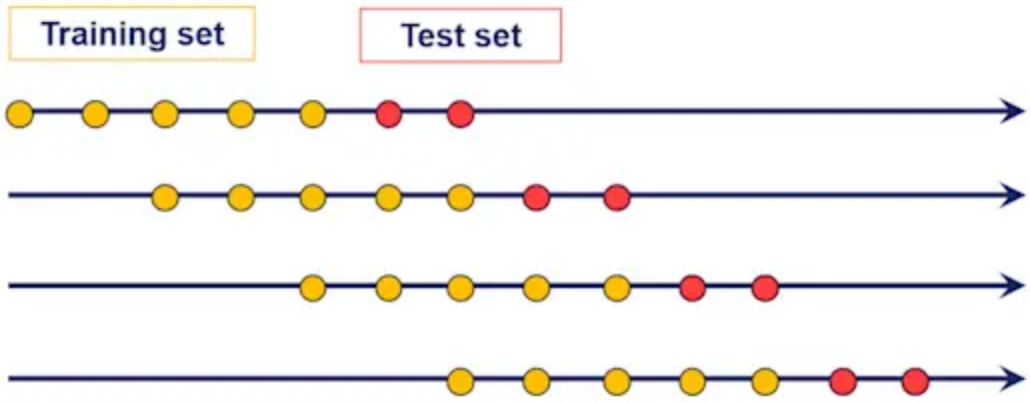
This is called a Rolling Forecast.



Sliding forecasts

The other type is that each time you cross-validate, you also trim the training set in the past while keeping the same size for the training data.

This is called a sliding forecast.



My preference goes for the rolling forecast.

Why? Well, if you're ever not using some data, it's normally because it is not worth it, and therefore it should not even be there for the modeling component. This fits the rolling model perfectly, and so it's my preference.

Let's show how this works now, by applying the rolling type during this project, in Python for the SARIMAX model.

Cross-validation for SARIMAX model

To start, you use the ARIMA function from the `pmdarima` library and you will no longer use the `auto_arima` function.

You create a dummy model with `p`, `d`, `q`, `P`, `D`, and `Q` parameters set, for instance, to 1.

(Don't worry, we will find the optimal values later).

Finally, don't forget to specify the seasonality. We have been using monthly data, so the seasonality parameter is `12`.

```
model = pm.ARIMA(order = (1,1,1),
                 seasonal_order = (1,1,1, 12),
                 X = exog_data,
                 suppress_warning = True,
                 force_stationarity = False)
```

Next, you specify the cross-validation settings.

For example, I prefer the rolling forecast, so I use the `RollingForecastCV` function.

Therefore, we now input what is the testing period for each Cross-Validation cycle (`h`), how much to add to the training data each time (`step`), and when to start testing the model (`initial`).

```
from pmdarima import model_selection
cv = model_selection.RollingForecastCV(h = 6,
                                       step = 1,
                                       initial = data.shape[0] - 24)
```

Finally, you put everything together.

Aside from the data, the model, and the cross-validation settings, you include the scoring error. I prefer the mean squared error, but you can find other options here.

```
cv_score = model_selection.cross_val_score(model,
                                           y = data,
                                           scoring = 'mean_squared_error',
                                           cv = cv,
                                           error_score = 1000000000000000)
```

Once you are done, you print the average errors using the numpy library:

```
error = np.sqrt(np.average(cv_score))
```

Parameter tuning to improve your model

Parameter tuning is key to going from a good Forecasting Product to a great one. Granted the programming can be challenging, but I know you can do it.

Why do we need to do parameter tuning? Innovation in analytics brings models that allow tailoring or, better yet, require it. The goal is to stop using one solution for every problem but rather optimize for each situation you have. That would bring the highest accuracy.

For the process, we start by defining the parameter options.

For instance, the `p` parameter can be `0`,`1`, or `2`. After determining, we run the model. Next, you measure the accuracy and save the error.

In a nutshell, it is nothing more than what we have done before. The difference is that we ran several models, each at a time, each with different parameters.

Parameter tuning for SARIMAX

The first step is to create a grid with all the options you want to try. In SARIMAX, there are 6 different parameters to tune: `p`, `d`, `q`, `P`, `D`, and `Q`.

```
from sklearn.model_selection import ParameterGrid
param_grid = {'p': [0,1],
              'd': [0,1],
              'q': [0,1],
              'P': [0,1],
              'D': [0,1],
              'Q': [0,1]}
grid = ParameterGrid(param_grid)
```

```
len(list(grid))
```

The logic for the Parameter Tuning is to build a model with each of the different variations specified above.

You would then use the Cross-validation pipeline built above since it is important to try and test the model in several circumstances and seasonal periods.

Finally, you store the error to compare later.

```
rmse = []

for params in grid:

  model = pm.ARIMA(order = (params['p'],params['d'],params['q']),
                   seasonal_order = (params['P'],params['D'],params['Q'], 7),
                   X = X,
                   suppress_warning = True,
                   force_stationarity = False)

  cv = model_selection.RollingForecastCV(h = 6,
                                         step = 1,
                                         initial = data.shape[0] - 24)
  cv_score = model_selection.cross_val_score(model,
                                             y = data,
                                             scoring = 'mean_squared_error',
                                             cv = cv,
                                             verbose = 2,
                                             error_score = 1000000000000000)


  error = np.sqrt(np.average(cv_score))
  rmse.append(error)
```

Now we can see which are the best parameters for the SARIMAX model.

```
tuning_results = pd.DataFrame(grid)
tuning_results['rmse'] = rmse
best_params = tuning_results[tuning_results.rmse ==
tuning_results.rmse.min()].transpose()
print(best_params)
```

And there you have it. Your SARIMAX model is tuned and cross-validated. Congrats on getting here!

SARIMAX Pros and Cons

Pros:

- The implementation is fairly easy to set up and use
- The above structure is the same for other forecasting models
- Even though SARIMAX is an old methodology, it still gets super good results

Cons:

- On the negative side, SARIMAX is also not great with long-duration time series, but I also think that it is not such a big deal. When a forecasting model is highly dependent on auto-regressive terms, then it usually does not deal well with long-term forecasts anyway
- Additionally, SARIMAX uses a simple linear regression for the regressors. Hence, if we have multicollinearity or non-linearity regressors, then SARIMAX will struggle
- Finally, SARIMAX does not allow more than one seasonality in the data. For instance, daily data has a weekly and monthly seasonality, which cannot be included in the SARIMAX modeling. Therefore SARIMAX is not ideal for daily data

Tl;dr

SARIMAX does the job, and it is one of those go-to forecasting models we all need.

Now it’s your turn

Hopefully, this guide helped to answer any common questions you had about ARIMA, SARIMA, and SARIMAX, and when to use them.

If you haven’t already, make sure to follow along with the code examples and have a play around, as it’s the best way to pick up and understand these models.

They might seem a little complex at first (especially if this is your first time running time-series forecasting), but they will get easier to use.